

Statistical Language Models

$$P(W) = P(w_1) \cdot P(w_2 | w_1) \cdot P(w_3 | w_1, w_2) \dots P(w_n | w_1, \dots, w_{n-1})$$

Theory · Worked Examples · Smoothing · Perplexity

Course

Natural Language Processing

Audience

Undergraduate students

What is a Language Model?

A probability distribution over sequences of words

Definition

A Language Model (LM) is a probabilistic model that predicts the likelihood of a sequence of words.

A **Language Model (LM)** assigns a probability $P(W)$ to a sentence $W = w_1 w_2 \dots w_n$

It tells us *how likely* a sequence is in the language. Higher $P(W)$ means the sentence is more natural / more grammatical / more typical of how people actually speak or write.

Intuition — Which is more likely?

✓ "the cat sleeps"
 $P(W) \approx \text{high}$ — natural English

✗ "sleeps the cat"
 $P(W) \approx \text{low}$ — wrong word order

✗ "cat the sleeps purple"
 $P(W) \approx \sim 0$ — nonsense

A good LM scores fluent text high and broken text low.

Statistical LMs estimate $P(W)$ directly from counts in a training corpus.

Why Language Models Matter

The probabilistic foundation behind almost every NLP system, useful in Estimate sentence probability, Generate natural language, Predict the next word

1 Disambiguation

Speech heard: “**peace**”, “**piece**”

Sentence:

“I want a **piece** of cake.” ✓, “I want a **peace** of cake.” ✗

The language model selects the more meaningful sentence.

2 Generation

Language models can generate new text by predicting the next word based on previous words (history).

Sampling from $P(w_i \mid \text{history})$ is how machines write text — from autocomplete to ChatGPT-style models.

3 Evaluation

Perplexity measures how “confused” a language model is while predicting the next word.

Perplexity gives a single number to compare any two LMs on the same test set — fair, fast, reproducible. Lower perplexity → better language model, Higher perplexity → weaker language model

4 Foundation

RNNs and Transformers learn the same $P(w_i \mid \text{history})$ — they just compute it with neural networks instead of counts.

Every modern LM — GPT, BERT, Llama — is a successor to the n-gram model you'll learn today.

Real-World Applications

Where language models show up in everyday technology

1

Speech Recognition

Pick the most likely transcription from acoustic candidates.

2

Machine Translation

Choose fluent target-language sentences over literal word-by-word output.

3

Spelling & Grammar

Detect and correct unlikely word sequences ("their/there/they're").

4

Autocomplete & Search

Suggest the next word, query completion, predictive keyboards.

5

Text Generation

Chatbots, story writing, summarization — sample from the LM.

Anatomy: Speech Recognition

Acoustic input → candidate transcripts → LM scores each → pick highest

"recognize speech"

$P = 0.0042$

WINNER

"wreck a nice beach"

$P = 0.0000017$

"recognise speak"

$P = 0.000028$

The LM never sees audio — it just ranks word sequences by $P(W)$.

The Chain Rule of Probability

Decompose a joint probability into a product of conditionals

$$P(w_1, w_2, \dots, w_n) = P(w_1) \cdot P(w_2 | w_1) \cdot P(w_3 | w_1, w_2) \dots P(w_n | w_1, \dots, w_{n-1})$$

Concrete example — $W = \text{"the cat sleeps"}$

$$P(\text{"the cat sleeps"}) = P(\text{the}) \cdot P(\text{cat} | \text{the}) \cdot P(\text{sleeps} | \text{the, cat})$$

Read it: "the" appears, then "cat" given we just said "the", then "sleeps" given we just said "the cat".

Exact and always true — just the definition of conditional probability applied repeatedly.

⚠ The problem

Computing $P(w_n | w_1, \dots, w_{n-1})$ directly is impossible — most long histories never appear in any corpus, so the count is 0. We need an approximation.

The Markov Assumption

Approximate the full history with just the last k words

$$P(w_n \mid w_1, \dots, w_{n-1}) \approx P(w_n \mid w_{n-k}, \dots, w_{n-1})$$

$k = 0$

Unigram

$P(w_i)$

Zero previous words. "sleeps" gets the same probability after "the cat" or "banana".

$k = 1$

Bigram

$P(w_i \mid w_{i-1})$

One previous word. Captures local order — "the cat" common, "cat the" rare.

$k = 2$

Trigram

$P(w_i \mid w_{i-2}, w_{i-1})$

Two previous words. Sharper grammar, but trigrams in test data are often unseen.

What is k ? The size of the context window — how many previous words the model conditions on. Note: **n -gram size = $k + 1$** (bigram has $k = 1$ because it conditions on 1 word, but the sequence has 2 words).

Trade-off: Higher $k \rightarrow$ more context but more sparsity. Most long histories never appear in any corpus.

Unigram Model

The simplest LM — assume every word is independent

Assumption

Every word is independent of all others — a pure "bag of words" view.

Sentence Probability

$$P(S) = P(w_1) \cdot P(w_2) \cdot P(w_3) \dots P(w_n)$$

Maximum Likelihood Estimate (MLE)

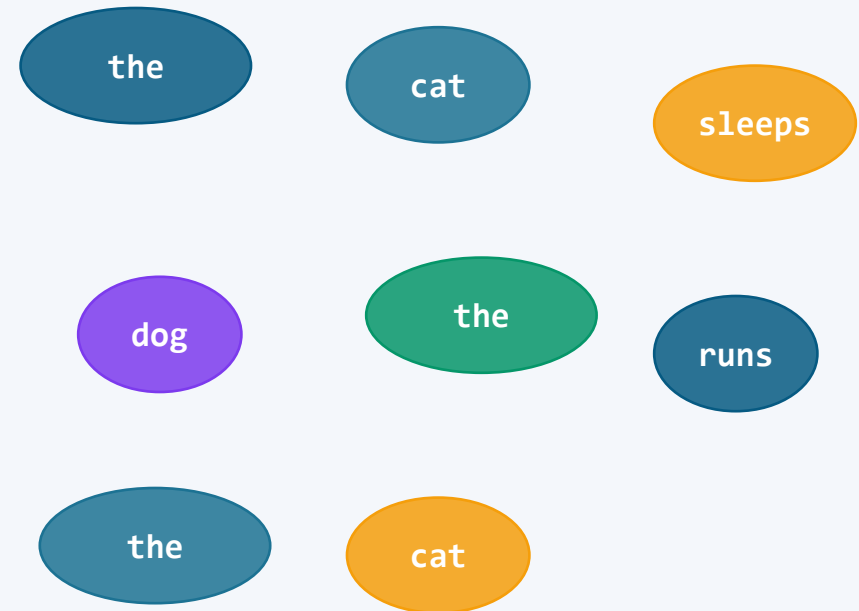
$$P(w) = \text{count}(w) / N$$

Pick probabilities that make the corpus most likely.
Mathematically, that just means: **count and divide**.

Strength · Weakness

Easy to estimate, but ignores word order entirely. "cat the sleeps" scores the same as "the cat sleeps".

Visualizing "bag of words"



Order is destroyed — all that remains are word frequencies.

Unigram Probability Estimation

Compute unigram probabilities from a 3-sentence corpus

Step 1 · Corpus

<s> I love NLP </s>

<s> I love deep learning </s>

<s> NLP is fun </s>

Step 2 · Tokens (excluding <s>, </s>)

I, love, NLP, I, love, deep, learning, NLP, is, fun

Total tokens **N = 10**

Counts: I=2, love=2, NLP=2, deep=1, learning=1, is=1, fun=1

Markers are bookkeeping, not real words — including them in N would inflate the denominator and shrink every word's probability.

Step 3 · Unigram probabilities

P(I)	P(love)	P(NLP)	P(deep)	P(learning)	P(is)	P(fun)
2/10	2/10	2/10	1/10	1/10	1/10	1/10
0.20	0.20	0.20	0.10	0.10	0.10	0.10

Step 4 · Probability of "I love NLP"

$$P(\text{I love NLP}) = P(\text{I}) \cdot P(\text{love}) \cdot P(\text{NLP}) = 0.20 \times 0.20 \times 0.20 = 0.008$$

0.008 = chance the model would generate this exact sequence by sampling words independently — not the probability the sentence appears in the corpus.

Bigram Model

Each word depends on exactly one previous word

$$\text{Sentence: } P(S) = \prod_i P(w_i \mid w_{i-1})$$

$$\text{MLE: } P(w_i \mid w_{i-1}) = \text{count}(w_{i-1}, w_i) / \text{count}(w_{i-1})$$

Sentence Markers

<s> marks the **start** of a sentence — used as the context for the first word.

</s> marks the **end** — lets the model learn what words typically end sentences.

Why bigrams help

They capture local word order — "the cat" is common, "cat the" is not. With just one word of context the model already distinguishes fluent text from scrambled text.

Trade-off

Sparser than unigrams — many valid word pairs never appear in training. (We'll fix this with smoothing.)

Bigram Probability — "I am Sam" Corpus

Show the formula → plug in counts → compute, for each probability

MLE formula: $P(w_i \mid w_{i-1}) = \text{count}(w_{i-1}, w_i) / \text{count}(w_{i-1})$

Corpus

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs </s>

count(<s>) = 3, count(I) = 3, count(am) = 2, count(Sam) = 2
Count (do, not, like, green, eggs) = 1

Bigram counts we'll need

(<s>, I), (I, am), (am, Sam), (Sam, </s>)

(<s>, Sam), (Sam, I), (I, am), (am, </s>)

(<s>, I), (I, do), (do, not), (not, like), (like, green), (green, eggs), (eggs, </s>)

count(<s>, I) = 2, count(I, am) = 2, count(am, Sam) = 1,
count(Sam, </s>) = 1, count(<s>, Sam) = 1, count(Sam, I) = 1

Each probability — formula, substitute, compute

$P(I \mid \text{<s>}) = \text{count}(\text{<s>}, I) / \text{count}(\text{<s>}) = 2/3 \approx 0.667$ $P(\text{Sam} \mid \text{<s>}) = \text{count}(\text{<s>}, \text{Sam}) / \text{count}(\text{<s>}) = 1/3 \approx 0.333$

$P(\text{am} \mid I) = \text{count}(I, \text{am}) / \text{count}(I) = 2/3 \approx 0.667$ $P(I \mid \text{Sam}) = \text{count}(\text{Sam}, I) / \text{count}(\text{Sam}) = 1/2 \approx 0.500$

$P(\text{Sam} \mid \text{am}) = \text{count}(\text{am}, \text{Sam}) / \text{count}(\text{am}) = 1/2 = 0.500$ $P(\text{am} \mid I) = \text{count}(I, \text{am}) / \text{count}(I) = 2/3 \approx 0.66$

$P(\text{</s>} \mid \text{Sam}) = \text{count}(\text{Sam}, \text{</s>}) / \text{count}(\text{Sam}) = 1/2 = 0.500$ $P(\text{</s>} \mid \text{am}) = \text{count}(\text{am}, \text{</s>}) / \text{count}(\text{am}) = 1/2 \approx 0.500$

$P(\text{<s> I am Sam </s>}) = (2/3) \cdot (2/3) \cdot (1/2) \cdot (1/2) = 4/36 = 1/9 \approx 0.1111$

Bigram Count Matrix

Rows = previous word w_{i-1} Columns = next word w_i

	I	am	Sam	do	not	like	green	eggs	</s>
<s>	2	0	1	0	0	0	0	0	0
I	0	2	0	1	0	0	0	0	0
am	0	0	1	0	0	0	0	0	1
Sam	1	0	0	0	0	0	0	0	1
do	0	0	0	0	1	0	0	0	0
not	0	0	0	0	0	1	0	0	0
like	0	0	0	0	0	0	1	0	0
green	0	0	0	0	0	0	0	1	0
eggs	0	0	0	0	0	0	0	0	1

How to read it

Cell (am, Sam) = 1

→ "am Sam" appeared once.

Row "I" sums to 3

→ count(I) = 3.

$P(\text{am} \mid \text{I}) = 2/3$

$P(\text{do} \mid \text{I}) = 1/3$

Lots of zeros — **the sparsity problem** we must address with smoothing.

To get probabilities, divide each row by its sum.

Trigram Model

Each probability shown as: formula $\rightarrow$ plug in counts $\rightarrow$ result

$$\text{MLE formula: } P(w_i \mid w_{i-2}, w_{i-1}) = \text{count}(w_{i-2}, w_{i-1}, w_i) / \text{count}(w_{i-2}, w_{i-1})$$

Corpus (double <s> = first trigram context)

<s> <s> the cat sat on the mat </s>
 <s> <s> the dog sat on the floor </s>
 <s> <s> the cat ran away </s>

Trigrams

Sentence 1: (<s>, <s>, the), (<s>, the, cat), (the, cat, sat), (cat, sat, on), (sat, on, the), (on, the, mat), (the, mat, </s>)

Sentence 2: (<s>, <s>, the), (<s>, the, dog), (the, dog, sat), (dog, sat, on), (sat, on, the), (on, the, floor), (the, floor, </s>)

Sentence 3: (<s>, <s>, the), (<s>, the, cat), (the, cat, ran), (cat, ran, away), (ran, away, </s>)

Trigrams Counts

Sentence 1: (<s>, <s>, the) = 3, (<s>, the, cat)=2, (the, cat, sat)=1, (cat, sat, on)=1, (sat, on, the) = 2, (on, the, mat) =1, (the, mat, </s>)=1

Bigram contexts: count(<s>, <s>) = 3, count(<s>, the) = 3, count(the, cat) = 2, count(cat, sat) = 1, count (sat,on)= 2, count (on, the) = 2, count (the mat) = 1

Each probability — formula, substitute, compute

$$P(\text{the} \mid \text{<s>, <s>}) = \text{count}(\text{<s>, <s>, the}) / \text{count}(\text{<s>, <s>}) = 3/3 = 1.000$$

$$P(\text{cat} \mid \text{<s>, the}) = \text{count}(\text{<s>, the, cat}) / \text{count}(\text{<s>, the}) = 2/3 \approx 0.667$$

$$P(\text{dog} \mid \text{<s>, the}) = \text{count}(\text{<s>, the, dog}) / \text{count}(\text{<s>, the}) = 1/3 \approx 0.333$$

$$P(\text{sat} \mid \text{the, cat}) = \text{count}(\text{the, cat, sat}) / \text{count}(\text{the, cat}) = 1/2 = 0.500$$

$$P(\text{ran} \mid \text{the, cat}) = \text{count}(\text{the, cat, ran}) / \text{count}(\text{the, cat}) = 1/2 = 0.500$$

$$P(\text{on} \mid \text{cat, sat}) = \text{count}(\text{cat, sat, on}) / \text{count}(\text{cat, sat}) = 1/1 = 1.000$$

More context = more sparsity. Most trigrams in any test set will be unseen — smoothing becomes essential.

Problems with MLE

Why "count and divide" alone is not enough for real language

1 Zero probability for unseen n-grams

Any bigram or trigram that didn't appear in training gets $P = 0$. One zero in the chain $\rightarrow P(W) = 0$ for the entire sentence.

2 Data sparsity

Vocabulary is huge (often $> 50,000$ words). Possible bigrams: V^2 , trigrams: V^3 . Even billion-word corpora miss most valid combinations.

3 Over-confidence in training

MLE puts ALL probability mass on observed events. It assumes the corpus is the whole language — wrong for any real dataset.

4 Perplexity becomes undefined

$PP(W) = P(W)^{-1/N}$. When $P(W) = 0$, perplexity $\rightarrow \infty$. The model is "infinitely surprised" and you can't compare it to anything.

Concrete: $P(\text{"the dog runs"}) = P(\text{the}|\langle s \rangle) \cdot P(\text{dog}|\text{the}) \cdot P(\text{runs}|\text{dog}) \cdot P(\langle s \rangle|\text{runs}) = \dots \cdot 0 \cdot \dots = 0$

Solutions: Smoothing Techniques

The core idea — steal a tiny bit of probability from seen events, give it to unseen ones

Core idea: Reserve a small slice of probability mass for events that didn't appear in training. *Total probability still sums to 1 — we just spread it more honestly.*

1

Laplace (Add-1)

Add 1 to every count. Simple, but over-smooths.

2

Add-k

Like Laplace, but with a smaller fraction k .

3

Witten-Bell

Use the count of distinct words seen after a context to estimate unseen mass.

4

Good-Turing

Use the count of n -grams that occurred once to estimate the count of zeros.

5

Linear Interpolation

Mix unigram, bigram, trigram with learned weights.

How to choose: *Laplace is for teaching. Add-k and Good-Turing are workhorse classics. Witten-Bell is competitive on real data. Interpolation is what production n -gram systems actually use — often combined with Good-Turing on each component.*

Laplace (Add-One) Smoothing

Pretend every possible bigram was seen at least once

$$P^*(w_i \mid w_{i-1}) = (\text{count}(w_{i-1}, w_i) + 1) / (\text{count}(w_{i-1}) + V)$$

Without smoothing ($V = 9$)

$$P(\text{am} \mid \text{I}) = 2/3 \approx 0.667$$

$$P(\text{Sam} \mid \text{I}) = 0/3 = 0$$

$$P(\text{do} \mid \text{I}) = 1/3 \approx 0.333$$

Zero probability $\rightarrow P(W)=0 \rightarrow \text{perplexity } \infty$

With Add-One ($V = 9$)

$$P^*(\text{am} \mid \text{I}) = (2+1)/(3+9) = 3/12 = 0.250$$

$$P^*(\text{Sam} \mid \text{I}) = (0+1)/(3+9) = 1/12 \approx 0.083$$

$$P^*(\text{do} \mid \text{I}) = (1+1)/(3+9) = 2/12 \approx 0.167$$

Every unseen bigram gets $\approx 0.083 \rightarrow$ no zeros, no infinities.

Caveat: Laplace tends to over-smooth — too much mass shifts to unseen events. Great for teaching, rarely used in production.

Laplace (Add-One) Smoothing — "I am Sam" Corpus

Add 1 to every count $\rightarrow$ divide by (count + V) · V = vocabulary size

$$\text{Laplace formula: } P(w_i \mid w_{i-1}) = [\text{count}(w_{i-1}, w_i) + 1] / [\text{count}(w_{i-1}) + V]$$

Vocabulary & unigram counts

Vocabulary (unique types predicted):

{ I, am, Sam, do, not, like, green, eggs, </s> }

$\rightarrow V = 9$

count(<s>)=3, count(I)=3, count(am)=2, count(Sam)=2

Bigram counts (numerator: count + 1)

count(<s>, I) = 2 $\rightarrow$ +1 = 3

count(I, am) = 2 $\rightarrow$ +1 = 3

count(am, Sam) = 1 $\rightarrow$ +1 = 2

count(Sam, </s>) = 1 $\rightarrow$ +1 = 2

Each smoothed probability — formula $\rightarrow$ substitute $\rightarrow$ compute

$$\begin{aligned} P(I \mid <s>) &= (\text{count}(<s>, I) + 1) / (\text{count}(<s>) + V) \\ &= (2 + 1) / (3 + 9) = 3/12 = 0.250 \end{aligned}$$

$$\begin{aligned} P(am \mid I) &= (\text{count}(I, am) + 1) / (\text{count}(I) + V) \\ &= (2 + 1) / (3 + 9) = 3/12 = 0.250 \end{aligned}$$

$$\begin{aligned} P(Sam \mid am) &= (\text{count}(am, Sam) + 1) / (\text{count}(am) + V) \\ &= (1 + 1) / (2 + 9) = 2/11 \approx 0.182 \end{aligned}$$

$$\begin{aligned} P(</s> \mid Sam) &= (\text{count}(Sam, </s>) + 1) / (\text{count}(Sam) + V) \\ &= (1 + 1) / (2 + 9) = 2/11 \approx 0.182 \end{aligned}$$

$$P(<s> I am Sam </s>) = (3/12) \cdot (3/12) \cdot (2/11) \cdot (2/11) = 36/17424 \approx 0.00207$$

Add-k Smoothing

A milder version: add a small fraction k instead of 1

$$P^*(w_i \mid w_{i-1}) = (\text{count}(w_{i-1}, w_i) + k) / (\text{count}(w_{i-1}) + k \cdot V)$$

Typical range: $0 < k < 1$. When $k = 1$ it's just Laplace.

Worked example · $k = 0.1$, $V = 9$

$$P^*(\text{am} \mid \text{I}) = (2 + 0.1) / (3 + 0.1 \cdot 9) = 2.1 / 3.9 \approx 0.538$$

$$P^*(\text{Sam} \mid \text{I}) = (0 + 0.1) / (3 + 0.9) = 0.1 / 3.9 \approx 0.026$$

Compared to Laplace ($k=1$), seen bigrams stay closer to MLE while unseen ones still get a non-zero share.

$k \rightarrow 0$

Closer to raw MLE, but unseen bigrams stay near zero.

$k = 0.1$

Mild smoothing — common practical choice.

$k = 1$ (Laplace)

Aggressive — over-smooths frequent words.

Linear Interpolation

Combine unigram, bigram, and trigram models with weights, Don't fully trust one model — combine multiple views of language.

$$P_{interp}(w_i | w_{i-2}, w_{i-1}) = \lambda_3 \cdot P(w_i | w_{i-2}, w_{i-1}) + \lambda_2 \cdot P(w_i | w_{i-1}) + \lambda_1 \cdot P(w_i)$$

with $\lambda_1 + \lambda_2 + \lambda_3 = 1$ and all $\lambda_i \geq 0$ (Sum of all λ typically = 1)

Worked example · $\lambda_1 = 0.1$, $\lambda_2 = 0.3$, $\lambda_3 = 0.6$

Given:

$$P(\text{sat} | \text{the, cat}) = 0.50$$

$$P(\text{sat} | \text{cat}) = 0.40$$

$$P(\text{sat}) = 0.05$$

$$\begin{aligned} P_{interp}(\text{sat} | \text{the, cat}) &= 0.6 \cdot 0.50 + 0.3 \cdot 0.40 + 0.1 \cdot 0.05 \\ &= 0.300 + 0.120 + 0.005 \\ &= \mathbf{0.425} \end{aligned}$$

Mass distribution



λ_3 trigram = 0.6

λ_2 bigram = 0.3

λ_1 unigram = 0.1

Witten-Bell Smoothing

Use the count of NEW words seen after a context to estimate unseen mass

Notation

$N(w_{i-1})$ = count(w_{i-1}) — total times context appeared

$T(w_{i-1})$ = number of distinct words that followed w_{i-1}

$Z(w_{i-1})$ = number of words that DID NOT follow it (= $V - T$)

Dataset

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs </s>

Example 1 — Seen Bigram Using Witten-Bell

We will compute:

$$P(am | I)$$

using Witten-Bell smoothing.

For Seen Bigrams

$$P(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{N(w_{i-1}) + T(w_{i-1})}$$

For Unseen Bigrams

$$P_{\text{unseen}}(w_i | w_{i-1}) = \frac{T(w_{i-1})}{Z(w_{i-1})(N(w_{i-1}) + T(w_{i-1}))}$$

Step 1 — Find All Continuations After “I”

From dataset: $I \rightarrow am$, $I \rightarrow am$, $I \rightarrow do$

Total occurrences of “I” $\rightarrow N(I) = 3$

because “I” appears 3 times.

Unique continuations after “I”

Observed next words: am , do $\rightarrow$ So: $T(I) = 2$

Step 5 — Substitute Values

Count(I, am)=2

$$P(am | I) = \frac{2}{3 + 2}$$

$$P(am | I) = 0.4$$

Witten-Bell Smoothing

Use the count of NEW words seen after a context to estimate unseen mass

Notation

$N(w_{i-1})$ = count(w_{i-1}) — total times context appeared

$T(w_{i-1})$ = number of distinct words that followed w_{i-1}

$Z(w_{i-1})$ = number of words that DID NOT follow it (= $V - T$)

Dataset

$\langle s \rangle$ I am Sam $\langle /s \rangle$

$\langle s \rangle$ Sam I am $\langle /s \rangle$

$\langle s \rangle$ I do not like green eggs $\langle /s \rangle$

Example 1 — UnSeen Bigram Using Witten-Bell

We will compute:

$$P(\text{green} \mid I)$$

using Witten-Bell smoothing.

Unseen continuations

Vocabulary: {I, am, Sam, do, not, like, green, eggs, $\langle /s \rangle$ }

$$Z(I)=7$$

For Seen Bigrams

$$P(w_i \mid w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{N(w_{i-1}) + T(w_{i-1})}$$

For Unseen Bigrams

$$P_{\text{unseen}}(w_i \mid w_{i-1}) = \frac{T(w_{i-1})}{Z(w_{i-1})(N(w_{i-1}) + T(w_{i-1}))}$$

Step 1 — Find All Continuations After “I”

From dataset: $I \rightarrow \text{am}$, $I \rightarrow \text{am}$, $I \rightarrow \text{do}$

Total occurrences of “I” $\rightarrow N(I) = 3$

because “I” appears 3 times.

Unique continuations after “I”

Observed next words: am , do $\rightarrow$ So: $T(I) = 2$

Step 5 — Substitute Values

Count(I,green)=0

$$P(\text{green} \mid I) = \frac{2}{7(3 + 2)}$$

$$P(\text{green} \mid I) = 0.057$$

Good-Turing Smoothing

Use the count of n -grams seen ONCE to estimate the count of n -grams seen ZERO times

Intuition: Things you saw once in training are good evidence for how many things you didn't see at all. Discount counts of frequent n -grams slightly, redistribute to unseen ones.

Notation & adjusted count formula

N_r = number of n -grams that occur exactly r times. (e.g. N_1 = how many bigrams appeared once)

Adjusted count: $r^* = (r + 1) \cdot N_{r+1} / N_r$

Probability: $P^*(n\text{-gram with count } r) = r^* / N$

Worked example · fish-catching corpus

You catch 18 fish total: 10 carp, 3 perch, 2 whitefish, 1 trout, 1 salmon, 1 eel.

$N_1 = 3$ (trout, salmon, eel — caught once) $N_2 = 1$ (whitefish) $N_3 = 1$ (perch) $N_{10} = 1$ (carp)

$P(\text{catching a NEW species next}) = N_1 / N = 3 / 18 \approx 0.167$

→ singletons predict the unseen-mass directly.

Adjusted count for trout ($r = 1$): $r^* = 2 \cdot N_2 / N_1 = 2 \cdot 1 / 3 \approx 0.667$

$P^*(\text{trout}) = 0.667 / 18 \approx 0.037$ (MLE was $1/18 \approx 0.056$ — discounted to free up mass for unseen)

Perplexity

How surprised the model is by a test set — lower is better

$$PP(W) = P(w_1 w_2 \dots w_n)^{-1/N}$$

Equivalent log form (numerically stable):

$$PP(W) = \exp\left(-\frac{1}{N} \cdot \sum_i \log P(w_i \mid \text{history})\right)$$

1 Branching factor

Perplexity = the weighted average number of equally likely choices the model considers at each word.

2 Lower is better

PP = 100 means the model is as uncertain as picking among 100 equally likely words. PP = 10 is much sharper.

3 Compare fairly

Always compute on a held-out test set — never on training data, where the model has memorized everything.

For a sentence $W = w_1, w_2, \dots, w_N$:

$$PP(W) = P(w_1, w_2, \dots, w_N)^{-\frac{1}{N}}$$

Using chain rule:

$$PP(W) = \left(\prod_{i=1}^N P(w_i | w_1, \dots, w_{i-1})\right)^{-\frac{1}{N}}$$

Log form (most commonly used):

$$PP(W) = \exp\left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i | \text{context})\right)$$

Typical bigram English: 100–300 Modern Transformers: 10–30 GPT-class on natural English: < 10

Computing Perplexity

Test sentence: $\langle s \rangle$ I am Sam $\langle /s \rangle$ · $N = 4$ predictions

Step 1 · Bigram probabilities

(values from Example 3.1)

$$P(I \mid \langle s \rangle) = 2/3$$

$$P(\text{am} \mid I) = 2/3$$

$$P(\text{Sam} \mid \text{am}) = 1/2$$

$$P(\langle /s \rangle \mid \text{Sam}) = 1/2$$

Step 2 · Joint probability

$$P(W) = (2/3)(2/3)(1/2)(1/2)$$

$$= 4/36 = 1/9$$

$$\approx \mathbf{0.1111}$$

Step 3 · Perplexity ($N = 4$)

$$PP(W) = \left(\frac{1}{9}\right)^{-1/4}$$

$$\approx \mathbf{1.732}$$

$PP \approx 1.732$ is misleadingly low — our corpus is tiny and every bigram appeared in training. Real-world bigram models on English typically score 100–300.

Complete Pipeline — One Worked Problem

Corpus → counts → smoothing → perplexity

Corpus

<s> the cat sleeps </s>
 <s> the dog sleeps </s>
 <s> the cat runs </s>

$V = \{\text{the, cat, dog, sleeps, runs, </s>}\}$, $|V| = 6$, $N = 12$

Bigram MLE

$P(\text{the} \mid \text{<s>}) = 3/3 = 1.00$
 $P(\text{cat} \mid \text{the}) = 2/3 \approx 0.667$
 $P(\text{dog} \mid \text{the}) = 1/3 \approx 0.333$
 $P(\text{sleeps} \mid \text{cat}) = 1/2 = 0.500$
 $P(\text{runs} \mid \text{cat}) = 1/2 = 0.500$
 $P(\text{sleeps} \mid \text{dog}) = 1/1 = 1.00$

Test sentence

<s> the dog runs </s>

$P(\text{the} \mid \text{<s>}) = 1.00$
 $P(\text{dog} \mid \text{the}) = 1/3$
 $P(\text{runs} \mid \text{dog}) = 0 \leftarrow \text{unseen!}$

Apply Laplace smoothing ($|V| = 6$) → no more zeros

Bigram	Smoothed formula	P^*
$P^*(\text{the} \mid \text{<s>})$	$(3+1)/(3+6) = 4/9$	0.444
$P^*(\text{dog} \mid \text{the})$	$(1+1)/(3+6) = 2/9$	0.222
$P^*(\text{runs} \mid \text{dog})$	$(0+1)/(1+6) = 1/7$	0.143
$P^*(\text{</s>} \mid \text{runs})$	$(1+1)/(1+6) = 2/7$	0.286

Joint & PP

$P^*(W) \approx 0.00403$
 $PP = 0.00403^{(-1/4)}$
 $\approx \mathbf{3.97}$

Smoothing turned an undefined infinite-perplexity case into a finite, meaningful score.

Try It Yourself

Solutions follow on the next slide

Reference corpus

`<s> she eats apples </s>` `<s> she eats oranges </s>` `<s> he eats apples </s>`

1 Bigram probabilities

Compute $P(\text{eats} \mid \text{she})$, $P(\text{eats} \mid \text{he})$, and $P(\text{apples} \mid \text{eats})$ from the reference corpus.

2 Sentence probability

Compute $P(\text{apples})$, then the probability of "`<s> she eats apples </s>`" under both unigram and bigram models.

3 Laplace smoothing

With $|V| = 6$, compute the smoothed $P^*(\text{oranges} \mid \text{he})$, which is unseen in training.

4 Perplexity

A bigram model gives "`<s> the dog barks </s>`":
 $P(\text{the} \mid \text{<s>}) = 0.5$, $P(\text{dog} \mid \text{the}) = 0.25$, $P(\text{barks} \mid \text{dog}) = 0.10$, $P(\text{</s>} \mid \text{barks}) = 0.40$.
Compute PP (use $N = 4$).

Practice Solutions

Step-by-step answers to all four problems

1 Bigram probabilities

$\text{count}(\text{she}, \text{eats}) = 2$, $\text{count}(\text{he}, \text{eats}) = 1$, $\text{count}(\text{eats}, \text{apples}) = 2$
 $\text{count}(\text{she}) = 2$, $\text{count}(\text{he}) = 1$, $\text{count}(\text{eats}) = 3$
 $P(\text{eats} | \text{she}) = 2/2 = 1.00$
 $P(\text{eats} | \text{he}) = 1/1 = 1.00$
 $P(\text{apples} | \text{eats}) = 2/3 \approx 0.667$

2 Sentence probability

$N = 12$, $\text{count}(\text{apples}) = 2 \rightarrow P(\text{apples}) = 2/12 \approx 0.167$
 Unigram: $P(\text{she}) \cdot P(\text{eats}) \cdot P(\text{apples})$
 $= (2/12)(3/12)(2/12) \approx 0.00694$
 Bigram (incl. $\langle s \rangle$, $\langle /s \rangle$):
 $(2/3) \cdot (1.00) \cdot (2/3) \cdot (1.00) = 4/9 \approx 0.444$

3 Laplace smoothing

$\text{count}(\text{he}, \text{oranges}) = 0$, $\text{count}(\text{he}) = 1$, $|V| = 6$
 $P^*(\text{oranges} | \text{he}) = (0+1)/(1+6)$
 $= 1/7 \approx 0.143$

4 Perplexity

$P(W) = 0.5 \times 0.25 \times 0.10 \times 0.40 = 0.005$
 $PP(W) = 0.005^{(-1/4)} = 1 / 0.005^{(0.25)}$
 $= 1 / 0.2659 \approx 3.76$

Key Takeaways

1

Higher-order n-grams capture more context

but suffer more from data sparsity — most long sequences never appear in training.

2

Smoothing is essential

Never deploy a pure MLE n-gram model — a single zero count breaks the whole sentence probability.

3

Evaluate with perplexity on held-out data

Training-set perplexity is misleading (the model has memorized those n-grams).

4

Perplexity is undefined when $P = 0$

Exactly the case smoothing is designed to prevent.

5

Modern LMs follow the same framework

RNNs and Transformers learn richer $P(w_i \mid \text{history})$ — they replace counts with neural networks.